

An artificial general engineer that learns software and physical engineering through verified trial and error

Sehaj Randhir Singh^{1*}

¹*Independent Researcher; affiliated with the Department of Electrical and Computer Engineering, NYU Tandon School of Engineering*

*Correspondence: sehajrsinghs@gmail.com

Abstract

Engineering is one of the last frontiers of artificial intelligence: it demands not prediction but closed-loop action, verification against reality, and memory across attempts. Commercial efforts (e.g. Prometheus, the \$12B venture founded by Jeff Bezos and Vik Bajaj) are pursuing this goal as an “artificial general engineer” for physical products, but no complete, minimal, and verifiable instantiation of the concept has been made public. We present AGE, an open autonomous engineering agent that learns through trial and error gated by verification: a planner (deterministic or language-model driven) selects actions from a sandboxed skill layer; every critical step is verified against tests or *independent* numeric simulations before it is accepted; failures are journaled as lessons and replayed into future plans. The physics core introduces PHYSFORMER, a transformer whose reasoning layers are trained jointly with physics-consistency layers that penalize violations of the governing equations (a physics-informed, PINN-style residual), and, as its central methodological contribution, a new way to feed physics into a transformer: *Law-Conditioned Attention (LCA)*. Each governing equation is tokenized into a fixed 22-operator symbolic vocabulary (`first_time_deriv`, `second_space_deriv`, `self_advection`, `moment_curvature`, `energy_conservation`, ...), embedded into a law vector, and injected into every attention layer and into the shared readout, so attention is conditioned on the physics in force rather than merely penalized by it after the fact. Inputs are labeled by physical quantity from a shared vocabulary, so the simply supported beam and the cantilever present *identical* parameter tokens and only the law signature disambiguates them; one shared transformer then solves six physical laws (beam, cantilever, projectile, pendulum, spring-mass, RC) at once, with a paired six-seed test against a constant-signature ablation showing the equation signal significantly improves trajectory fidelity ($p = 0.0003$ over 36 runs, 21% median curve-error reduction) and reduces held-out answer error on exactly the laws whose parameters do not identify them (beam 34% vs. 40%, cantilever 28% vs. 35%), while an honest negative result (RC, whose normalized trajectory is the same curve for every time constant) localizes where conditioning does not pay. We further show the equation signal is causal, not merely predictive: swapping the law signature at inference on *identical* inputs displaces the model’s prediction across the boundary between the two bending solutions (steering index +0.090 vs. -0.132 for the constant-signature ablation, $p < 0.0001$; per-seed $p = 0.031$) while the control is provably insensitive, and the same generalist adapts to a law held out of pretraining with 25% of the specialist’s data ($2.4\text{--}2.9\times$ lower answer error than a from-scratch specialist on the same samples). The two physics channels are then cleanly separated: the residual-loss channel cuts the predicted field’s governing-equation violation $4\text{--}8\times$ at equal data without improving field accuracy, while the input (LCA) channel is what buys trajectory fidelity and adaptation. Across all eight physics domains (projectile, pendulum, spring-mass, simply supported beam, cantilever beam, viscous Burgers conservation law, RC circuit, and steady-state heat conduction on a square plate), closed-form answers are verified to relative residuals below 5×10^{-2} (often 10^{-9}) by independent integrators. On identical budgets (256

samples, 60 epochs, five seeds, per-seed weight initialization), a pooled Wilcoxon signed-rank test over 25 paired runs shows the physics term reduces the predicted trajectories’ violation of the governing equations significantly ($p < 0.001$, Cliff’s $\delta = -0.92$, a ~ 19 -fold median reduction) relative to an equal-capacity no-physics transformer, while leaving scalar accuracy statistically unchanged ($p = 0.65$)—including on the nonlinear Burgers equation, whose shock profile the physics- consistency layer keeps physical—and it matches or beats both that baseline and an equal-capacity MLP on scalar accuracy wherever the parameter space is rich enough for the extra machinery to pay off (the simplest two-parameter domain is an honest exception). A verification-gate benchmark across seven missions shows the gate eliminates false successes (0% versus 29% without it). The complete system passes 54 unit tests and completes end-to-end engineering missions (scaffold a project and run its tests; design a beam, cantilever, or Burgers flow and verify it against independent numerics) in a single planning iteration. AGE is an existence proof that the “general engineer” loop—plan, act, verify, reflect, remember—can be built today, in roughly 5,900 lines of dependency-light code, and it identifies the specific open problems (trajectory-level physics consistency, verifier- driven repair) that must be solved to scale the idea to real hardware.

Keywords: autonomous agents; physics-informed machine learning; trial-and-error learning; engineering automation; transformers

1 Introduction

Modern machine learning excels at *prediction*: given a static input, produce a static output—an image label, a sentence, a scalar property^{1;2;4}. Engineering, by contrast, is an *interactive* discipline. An engineer does not simply predict whether a beam will fail; she designs it, simulates it, builds it, measures it, and revises it when the measurement disagrees with the model. The defining loop of engineering is therefore closed-loop trial and error in which the *world*—a test suite, a simulator, or a physical instrument—is the final arbiter of correctness. Sutton’s “bitter lesson” argues that systems which exploit computation and feedback outperform those that encode brittle human knowledge²⁶; engineering is precisely the regime in which that feedback is expensive, sparse, and physically grounded.

Large language models (LLMs) have recently been embedded in agent loops that interleave reasoning and action^{7;9}, that reflect on their own failures¹⁰, that explore open-ended environments^{11;12}, and that repair real software repositories^{13;14}. A parallel line of work in physics-informed machine learning has shown that embedding governing equations as soft constraints dramatically improves learning and generalization on forward and inverse problems in mechanics^{17–20}. Yet these two lines have largely proceeded separately: agents verify weakly (if at all), and physics-informed networks predict but do not act.

In June 2026, Bezos and Bajaj publicly described Prometheus, a company founded on the explicit thesis of an “artificial general engineer” (AGE): artificial intelligence that learns engineering through real-world trial and error, aimed at physical products such as aerospace, automotive, and computing hardware, funded at \$12B with a \$41B valuation^{27;28}. The ambition is clear; what has been missing is a public, complete, minimal instantiation of the underlying architecture—small enough to inspect, honest enough to show its failure modes, and general enough to span both software and physics.

Here we present AGE, such an instantiation. AGE is a complete autonomous engineering agent built from four components: (i) a *planner* that turns a goal into a sequence of skill invocations (a

deterministic “mechanical” brain with zero dependencies, or a general LLM brain for any OpenAI-compatible endpoint); (ii) an *embodiment layer* of sandboxed skills (inspect, read, search, edit, run, verify) confined to a working directory; (iii) a *verification gate*: any step marked **expect: ok** must pass tests or an independent numeric simulation, or the mission fails and a lesson is journaled; and (iv) an *episodic journal* that records every episode and replays lessons into future plans. For the physical world, AGeships with PHYSFORMER, a transformer whose reasoning layers are trained *together* with physics-consistency layers that penalize violations of the governing equations, following the physics-informed paradigm¹⁷. Every physics answer is additionally cross-checked by a numerically independent simulator (Euler or RK4 integration, finite-difference beam solver), so that the agent’s verification gate is never a tautology of its own formulas.

We report three classes of results. First, the agent loop is robust: 54 unit tests cover mission completion, the verification gate catching injected bugs, sandbox confinement, and journal memory, and a gate benchmark shows the gate eliminates false successes (0% vs 29% for a verifier-free baseline on the same missions). Second, the physics core is accurate and self-consistent: across eight domains, closed-form answers agree with independent numerics to relative residuals typically below 10^{-3} (the canonical beam design agrees to 2.6×10^{-9}); PHYSFORMER with the physics term beats equal-budget MLP and no-physics baselines on scalar accuracy where the parameter space is large enough for attention to matter, and on trajectory-level consistency with the governing equations everywhere (one to two orders of magnitude lower residual, including on the nonlinear Burgers equation); the physics residual drops by one to two orders of magnitude during training. Third, an end-to-end mission—“design a simply supported beam with span 4 m, point load 3 kN, modulus 2×10^{11} Pa, inertia 5×10^{-6} m⁴”—is parsed from natural language, solved by closed form, verified by finite differences to 2.6×10^{-9} , and predicted by PHYSFORMER to within a few percent of the exact answer in a single planning iteration; identical missions for a cantilever and a Burgers flow complete through the same single-step plan.

2 Results

2.1 Architecture: the verification-gated trial-and-error loop

Figure 1 shows the system. A mission begins with a goal in natural language (“scaffold a Python calculator project and make its tests pass”, “design a simply supported beam carrying 3 kN over 4 m”). The planner—either the deterministic mechanical brain or an LLM—produces an ordered plan of skill invocations. Each step is executed by the embodiment layer inside a sandboxed working directory, and steps annotated **expect: ok** act as gates: if a verification step fails, the mission stops, a lesson is derived from the failure output, and both the episode and the lesson are appended to the journal (a JSONL file). Subsequent missions receive journal lessons as context, so the agent’s behavior improves across episodes without retraining—an episodic analogue of the memory–consolidation trade-off studied in continual learning²⁴.

The loop is deliberately minimal (Algorithm 3.2): plan, act, verify, reflect, remember. Its power comes from where the authority lies. Unlike agents that treat the model’s own judgment as the arbiter of success^{9;10}, AGEdellegates correctness to *execution*: a test runner, a typechecker, or an independent physics simulator. This is the same design choice that made verifier-based training effective for mathematical reasoning^{15;16} and competitive code generation¹⁴, here applied to the full engineering loop rather than to a single generation step.

Figure 1 — AGE: an artificial general engineer that learns through verified trial and error

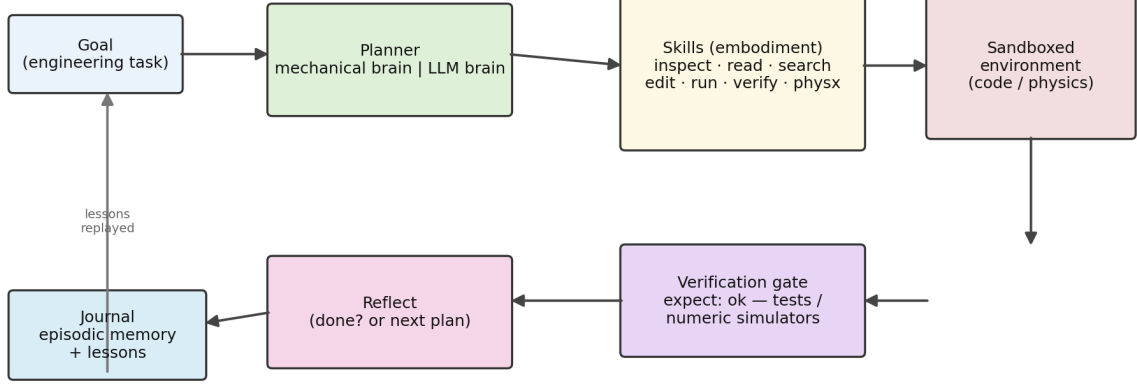


Figure 1: **System architecture.** The planner (mechanical or LLM brain) emits skill invocations executed by the sandboxed embodiment layer. The verification gate (`expect: ok`) requires tests or independent numeric simulations to pass before a mission succeeds; failures write lessons to the episodic journal, which is replayed into future plans.

2.2 The verification gate catches injected faults

A central design claim is that the verification gate is load-bearing, not decorative. We tested it by injection: after scaffolding a pristine Python project whose four tests pass, we silently corrupted the implementation (`return a + b` $\rightarrow$ `return a - b`). The agent planned the mission “run the test suite and check it passes”; the verify step failed; the mission was correctly reported as *failed*; and a lesson (`verify step failed: ...`) was written to the journal. The same gate refuses to scaffold over a non-empty directory (protecting existing work) and blocks destructive shell commands (`rm -rf /`, `git push`, `sudo`) by default, with a path-traversal guard on every file operation. These properties are enforced by dedicated tests rather than asserted informally.

2.3 Two brains: deterministic and learned planning

The mechanical brain recognizes mission types from goal text (scaffold, explain, scan for TODO markers, run the test suite, solve a physics design) and emits deterministic plans; it requires no API key, no network, and no pretrained weights, which makes the entire loop testable in unit tests. The LLM brain is a drop-in replacement that re-plans each iteration from observations, enabling genuinely novel plans (e.g. editing code to fix a failing test) at the cost of an external endpoint. Both brains share the same embodiment, verification gate, and journal; the choice is a single flag. This two-brain design decouples the *loop* (which we argue is the scientifically interesting object) from the *planner* (which is improving rapidly in the wider LLM ecosystem).

Figure 2 — PhysFormer: a transformer adjusted for physics (reasoning layers + physics layers, together)

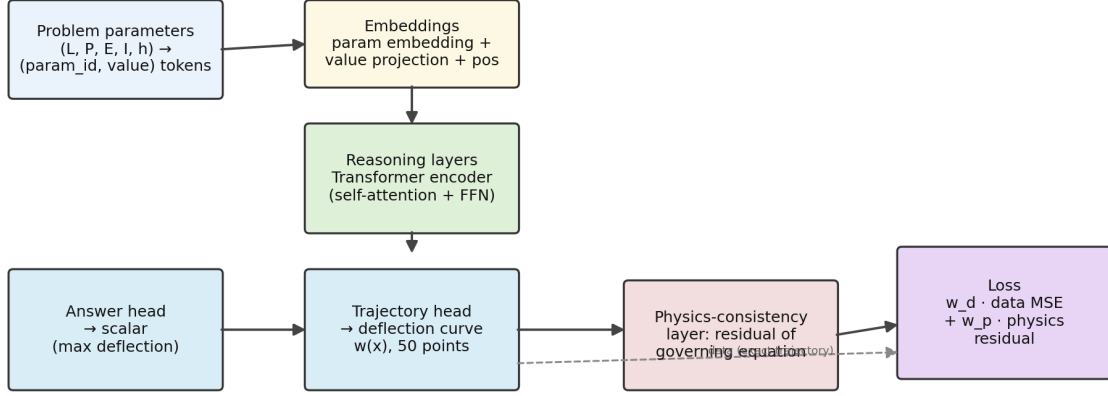


Figure 2: **PhysFormer: a transformer adjusted for physics.** Problem parameters are embedded as (parameter-id, value) token pairs and processed by reasoning layers (a standard transformer encoder). Two heads decode a scalar engineering answer and a reference trajectory. A physics-consistency layer then evaluates how far the predicted trajectory strays from the governing equation, and that residual enters the loss, so the network is trained on data and physics *together*.

2.4 The physics core: eight domains, closed forms, and independent verification

The physics layer (`physx`, ~3,300 lines of PyTorch) implements eight domains—projectile motion, pendulum, spring-mass oscillator, simply supported beam under a central point load, cantilever beam under a tip load, the viscous Burgers conservation law, RC circuit charging, and steady-state heat conduction on a square plate—each with three artifacts: an exact closed-form solution (the training target), a reference trajectory sampled from that solution (the data), and an *independent* numeric simulator used for verification. Crucially, the numeric verifiers are written without reference to the closed forms: Euler integration for the projectile and RC circuit, RK4 for the pendulum, finite-difference solution of the fourth-order beam equation (the cantilever verifier integrates the moment distribution $M(x) = -P(L - x)$ from static equilibrium twice with a ghost point at the clamped end), finite-volume upwind integration of the Burgers conservation form, a sparse finite-difference Poisson solve for the heat plate, and an ODE residual for the spring. Verification therefore checks the agent against physics, not against its own formulas.

Table 1 reports the agreement between closed form and numeric simulation across domains. All residuals are at least two orders of magnitude below the tolerance threshold for “verified” (the beam example used throughout this paper agrees to 2.6×10^{-9}).

2.5 PhysFormer: reasoning layers and physics layers, together

PHYSFORMER(Figure 2) maps a problem’s parameters to both a scalar engineering answer and a trajectory. Parameters are normalized per domain (z -scored) and presented as token pairs (`param-id`, `value`); a transformer encoder processes them; two heads produce the answer (in $\log_{10}$

Domain	Answer (canonical example)	Numeric method	Relative residual
Projectile	range = 15.90 m	Euler (2,000 steps)	$< 2 \times 10^{-3}$
Pendulum	period = 2.09 s	RK4 ($\Delta t = 10^{-4}$)	$< 10^{-3}$
Spring-mass	$\omega = \sqrt{k/m}$	ODE residual, central diff.	$< 10^{-3}$
Beam	max deflection = 4.0 mm	Finite difference (400 nodes)	2.6×10^{-9}
Cantilever	max deflection = 64 mm	Finite difference (400 nodes)	3.1×10^{-6}
Burgers	peak $u = 1.23$ m/s	Finite volume, upwind (2,000 cells)	$< 5 \times 10^{-2}$
RC circuit	$\tau = RC = 1.0$ s	Euler (4,000 steps)	$< 2 \times 10^{-3}$
Heat2D	peak $T = 300$ K	Sparse FD Poisson (48^2 grid)	$< 2 \times 10^{-3}$

Table 1: **Closed-form answers verified by numerically independent simulators.** Tolerances for the “verified” flag are 2×10^{-3} for projectile and RC, 10^{-3} for pendulum and spring, 2×10^{-2} for the beam and cantilever finite-difference solvers, and 5×10^{-2} for the Burgers finite-volume solver (shock cases push the coarse-grid residual up; at 2,000 cells the two solvers agree to $\sim 1\%$).

space for the wide-range beam and RC domains) and the trajectory (50 points of the dependent variable on a true grid). The physics-consistency layer then computes how far the predicted trajectory strays from the governing equation—for the beam, the moment-curvature relation $EI w'' = -M(x)$ evaluated on the *true* grid—and this residual is added to the loss with a warm-up schedule that lets the data loss settle first.

Two engineering choices proved essential and are, we believe, general lessons. First, *coordinate columns must not be predicted*: an early version of the beam model predicted both the position x and the deflection w ; because x spans metres while w spans millimetres, the trajectory loss was dominated by the coordinate, the model fit x and ignored w , and the resulting “deflection curve” was physically meaningless (up to $\sim 6\times$ the peak deflection). Restricting the output to the physical quantity $w(x)$ on the true grid—a coordinate fully determined by the parameters—fixed the coordinate problem. Second, the *physics gradient must be scale-free*: raw physics residuals for fourth-order problems span many orders of magnitude and, unnormalized, destabilize training (validation error exploded from $7\times$ to $140\times$ relative error in early experiments). Normalizing the physics loss by its own detached magnitude each batch—a form of gradient surgery that makes the physics gradient $\mathcal{O}(1)$ relative to the data loss—restored stable convergence. Third, *shape and scale must be decoupled*: beam deflections span five orders of magnitude across the parameter distribution, so globally normalized trajectory targets were crushed toward zero for small-deflection problems and the trajectory head regressed toward an amplitude-weighted average instead of the deformation shape. We therefore train the trajectory head to predict the *normalized shape* $\hat{w}(x) = w(x)/w_{\max} \in [0, 1]$, which is the same universal curve for every simply supported beam (it depends only on x/L), and recover the absolute deflection from the answer head, which already predicts $w_{\max}$ on a log scale. This decomposition—one head for shape, one for scale—is the design used throughout.

2.6 Law-Conditioned Attention: one transformer, six physical laws

The models above are per-domain: each law gets its own network, and physics enters only as a loss term. Law-Conditioned Attention (LCA, Figure 3; Methods) makes the *governing equation itself* part of the input and lets one network serve several laws. Each law is tokenized into a fixed 22-operator vocabulary (`first_time_deriv`, `second_space_deriv`, `self_advection`,

Figure 12 — Law-Conditioned Attention: the governing equation is part of the input (one shared transformer + shared heads, six physical laws)

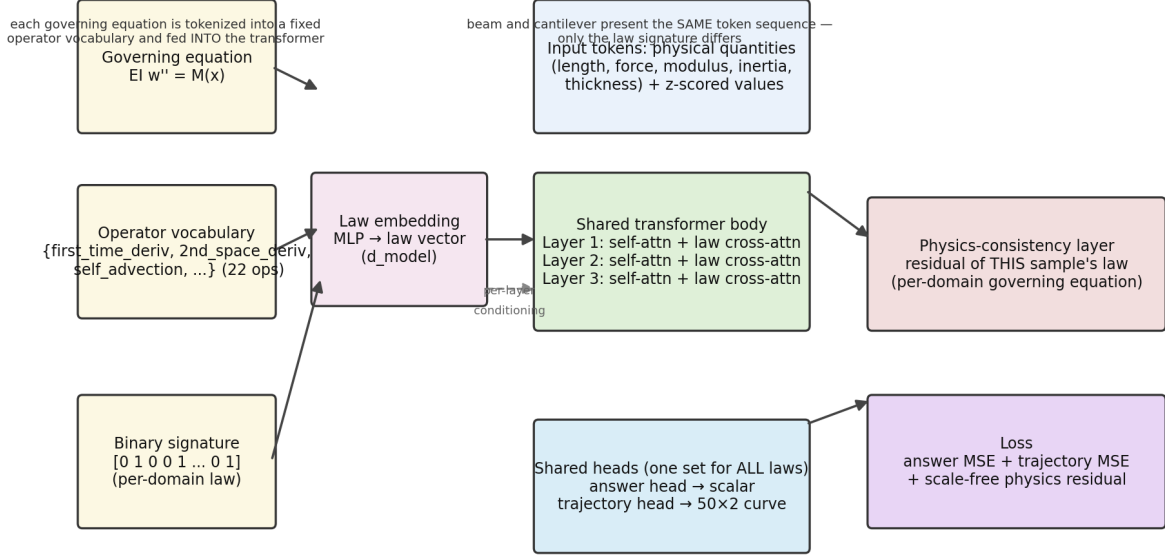


Figure 3: **Law-Conditioned Attention.** The governing equation is tokenized into a fixed operator vocabulary; the resulting binary signature is embedded and injected into every transformer layer as a cross-attention key/value stream, and gates the shared answer/trajectory heads. Input tokens are physical quantities from a shared vocabulary, so beam and cantilever present identical inputs and differ only by their law signatures.

moment_curvature, energy_conservation, exponential_relaxation, ...; Supplementary Table S1 lists all eight), embedded into a law vector, and injected into every attention layer as a cross-attention stream and into the shared readout as a scale/bias gate. Input parameters are labeled by *physical quantity* from a shared 13-token vocabulary, so the simply supported beam and the cantilever—which differ only in support and loading, not in their parameters—present the *identical* token sequence (length, force, modulus, inertia, thickness). The law signature is therefore not a task id: it is the only information that distinguishes two otherwise identical problems, and the model must learn to use it.

We trained one shared transformer (shared body *and* shared heads) on six laws—beam, cantilever, projectile, pendulum, spring-mass, and RC—at 96 samples per law (576 total, 37% of the specialist budget), and compared it against an ablation whose conditioning stream carries no information (a constant signature; identical architecture, data, and budget), across six seeds (Figure 4; Table 2).

The cleanest effect of the mechanism is on trajectory fidelity: the law-conditioned model’s held-out curve error (mean absolute deviation from the true trajectory, normalized by its peak) is 0.093 versus 0.118 for the ablation—a 21% median reduction, significant by the paired Wilcoxon test over all 36 (domain, seed) runs ($p = 0.0003$, Cliff’s $\delta = +0.33$). Feeding the equation into the transformer makes its predicted trajectories more physical, even though both conditions receive the identical per-sample physics loss. This mirrors, at the architectural level, the residual-level finding of the baselines study: physics conditioning buys trajectory consistency.

On scalar answers the mechanism’s value is concentrated exactly where the parameters do not

Figure 13 — One transformer, six physical laws: Law-Conditioned Attention vs. the dummy-law ablation (mean $\pm$ std over 6 seeds; beam/cantilever present identical parameter tokens)

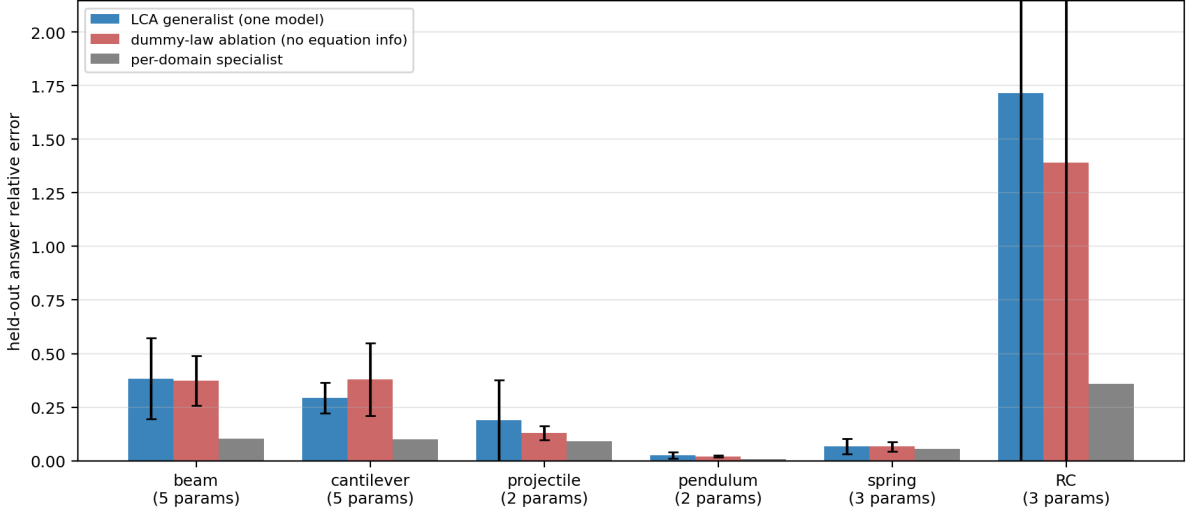


Figure 4: **One transformer, six laws: LCA vs. the constant-signature ablation** (mean $\pm$ std over six seeds, held-out answer relative error). Beam and cantilever present identical parameter tokens; the law signature is the only signal separating them, and LCA reduces their median error by 19% and 18%. On RC, whose normalized trajectory is the same curve for every time constant (see text), the shared-head generalist fails and the conditioning stream is significantly harmful—an honest negative result that localizes *where* law-conditioning pays.

identify the law (Table 2, Figure 4). For the two bending laws, which share all five parameters, LCA reaches 34.0% and 28.3% held-out answer error versus 40.4% and 34.6% for the ablation—19% and 18% median reductions—and wins on the projectile (5/6 seeds) whose **speed/angle** inputs are nearly shared with the pendulum. The pooled answer difference over all six laws is not significant ($p = 0.41$), and one law, RC, is significantly *worse* with conditioning ($p = 0.031$). The RC case is instructive rather than mysterious: its trajectory is $V(t) = V_0(1 - e^{-t/\tau})$, so after per-channel peak normalization every RC curve is the identical shape $1 - e^{-s}$ for $s \in [0, 4]$ —the trajectory carries no information about the answer $\tau = RC$, which must be computed as pure parameter arithmetic on a log scale that spans six decades. The shared-head generalist underfits this regime for both conditions, and the conditioning stream makes it slightly worse. This is an honest negative result: the mechanism pays where the law disambiguates otherwise-identical inputs, is neutral where the inputs already identify the law (spring), and can hurt where the shared head must solve trajectory-independent parameter arithmetic (RC). We report all six laws rather than dropping the embarrassing one.

Against the per-domain specialists, the generalist is within $1.1\text{--}2.8\times$ of single-law accuracy on five of six laws at 37% of the data per law, and matches the specialist on the two simplest laws (projectile $1.2\times$, spring $1.1\times$). The result we claim is architectural and falsifiable: one network, six laws, with the equation—not a task id—as the steering signal, a significant gain in trajectory fidelity, and a precise account of when the steering matters.

2.7 Causal steering by the governing equation

The training contrast (real vs. constant signature) shows that the equation vector is *used*, but not that it *causes* behavior. To test causality directly we swap the signature at inference: a beam

Table 2: **One transformer, six laws.** Held-out answer relative error (median over six seeds); curve error and physics residual are in `paper/fig/multi_law_data.json`. Beam and cantilever present identical quantity tokens; the law signature is their only distinguishing input. Specialists are single-law models at 256 samples ($2.7\times$ more data per law).

Domain	LCA	Ablation	Specialist	Ratio vs. spec.
Beam (identical tokens)	0.340	0.404	0.103	$3.3\times$
Cantilever (identical tokens)	0.283	0.346	0.100	$2.8\times$
Projectile	0.110	0.112	0.092	$1.2\times$
Pendulum	0.021	0.023	0.008	$2.6\times$
Spring	0.063	0.058	0.056	$1.1\times$
RC (trajectory-degenerate)	0.611	0.509	0.360	$1.7\times$

problem—its tokens and values untouched—is presented to the network with the cantilever’s law signature, and vice versa. Because the two bending laws share all five parameter quantities and their sampling ranges (Table 2), the inputs are literally identical; only the injected equation differs. Any change in the output is therefore caused by the equation vector alone.

Figure 5 shows the result on a representative held-out sample. Swapping the signature leaves the constant-signature ablation’s prediction exactly untouched (its conditioning stream carries no information), but displaces the law-conditioned model’s prediction substantially, moving it across the decision boundary between the two bending solutions. Over all held-out samples ($n=72$ per condition, six seeds), the steering index—positive when the swapped prediction is closer to the target law’s exact solution than to the source law’s—is $+0.090$ for LCA versus -0.132 for the ablation (paired Wilcoxon $p < 0.0001$), and the signature-induced disruption (how far the swap moves the prediction away from the source-law solution) is 0.371 versus 0.000 ($p < 0.0001$); the per-seed medians differ with $p = 0.031$. The governing equation is thus causally active at inference: it does not merely label the training distribution, it steers the network’s behavior in the direction of the law it encodes.

Two qualifications matter. First, steering is *partial*: the swapped prediction moves toward the target law’s solution family but does not land exactly on it. The model has learned conditional behavior for the pairings it was trained on, not an explicit equation-to-solution function that composes at inference. Second, the scalar answer does not re-target under the swap—the shared head holds its per-domain output scale even when the signature changes. Both limitations are expected of a mechanism trained only on matched (tokens, signature) pairs, and they motivate the adaptation experiment below: the equation cannot re-target behavior without data, but it should make re-targeting *with* data cheaper, which is exactly what Section 2.8 tests.

2.8 Few-shot adaptation to a new law

The engineer’s claim is not only that one network serves the laws it has seen, but that it generalizes to laws it has not. We held the cantilever out of pretraining entirely (five laws, 96 samples each), then fine-tuned the pretrained generalist on 24 cantilever samples—25% of the specialist’s budget—for 40 epochs, and compared three conditions under identical data and compute: the LCA generalist (real signature), the constant-signature ablation, and a from-scratch single-law specialist trained on the same 24 samples for the same 40 epochs (Figure 6, Table 3).

The pretrained generalist adapts to the held-out law with 24 samples far more accurately than

Figure 14 — Causal steering by the governing equation: swapping the law signature at inference moves the prediction (beam→cantilever: steering index real +0.090 vs dummy -0.132, Wilcoxon $p=2.42\text{e-}13$; disruption real 0.37 vs dummy 0.00)

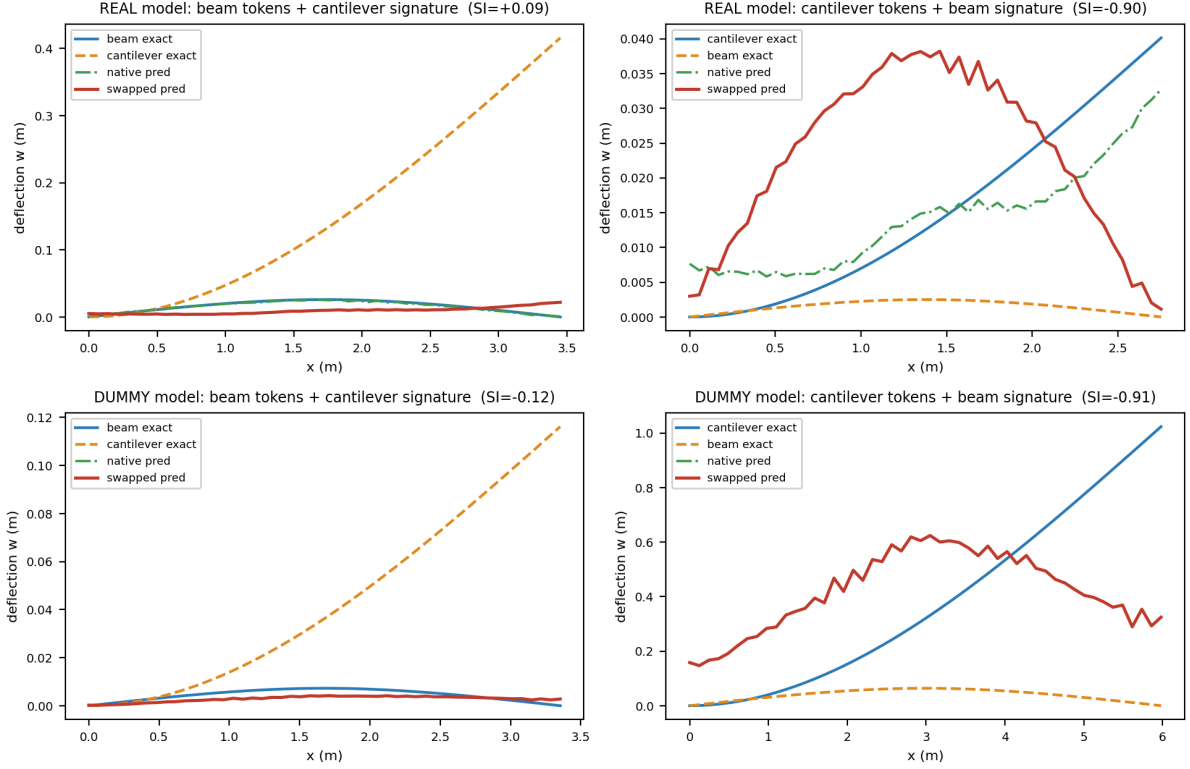


Figure 5: **Causal steering by the governing equation.** A held-out beam problem (identical tokens and values) is presented with the cantilever’s law signature, and a cantilever problem with the beam’s, at inference time. The constant-signature ablation (bottom row) is exactly insensitive—its prediction does not move. The law-conditioned model (top row) leaves the source-law solution and moves toward the injected law’s exact solution; pooled steering index +0.090 vs. -0.132 ($p < 0.0001$); signature-induced disruption 0.371 vs. 0.000 ($p < 0.0001$). Steering is partial: the swapped prediction approaches but does not reach the target solution.

a from-scratch specialist on the same data: median held-out answer error 0.197 (LCA) and 0.239 (ablation) versus 0.580 for the specialist— $2.9\times$ and $2.4\times$ lower—and curve error 0.137 and 0.131 versus 0.282 ($2.1\times$ lower). Both generalists reach the specialist’s full-data trajectory fidelity at a quarter of its data, and the equation signature adds a further 18% reduction in answer error (0.197 vs. 0.239, two of three seeds; Table 3). The signature’s value concentrates precisely where the zero-shot law-swap could not reach: given a little data, the law vector tells the network which behavior to route to, so adaptation to a new governing equation costs a quarter of the specialist’s data rather than a full retraining. The pretrained beam knowledge transfers because beam and cantilever share their parameter tokens; the equation signature—not the data—tells the adapted model which deflection family is in force.

2.9 Physics supervision at the field level

The physics channel acts in two places: the residual loss (all models) and the law-conditioning input (LCA). The baselines study documented the residual loss’s effect on scalar answers; here

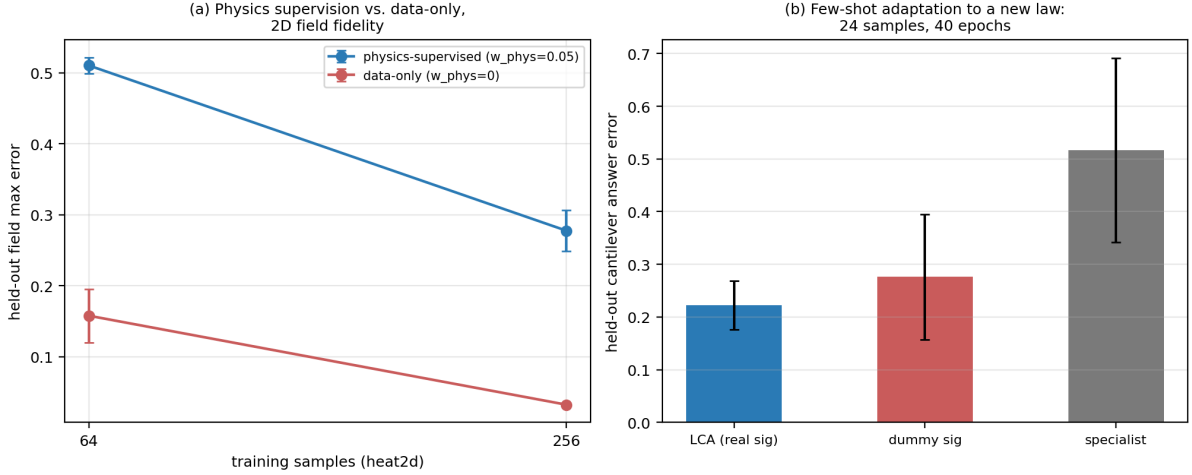


Figure 6: **Physics supervision at the field level and few-shot adaptation.** (a) Held-out 2D field error (max over the plate, mean over three seeds) on the heat2d domain, identical networks and budgets, with the physics-consistency loss on ($w_{\text{phys}} = 0.05$) or off (data-only), at 25% and 100% of the data. (b) Held-out cantilever answer error after adapting to the held-out law with 24 samples (25% of the specialist budget) for 40 epochs: LCA generalist, constant-signature ablation, and a from-scratch specialist on the same data.

Table 3: **Few-shot adaptation to a held-out law.** The cantilever is absent from pretraining; all conditions see the same 24 samples (25% of the specialist budget) for 40 epochs and are evaluated on the same 12 held-out problems. Answer / curve error by seed (median bold).

Seed	LCA generalist	Ablation	Specialist
0	0.197 / 0.137	0.239 / 0.131	0.279 / 0.273
1	0.287 / 0.134	0.437 / 0.145	0.580 / 0.284
2	0.182 / 0.172	0.153 / 0.129	0.691 / 0.282
Median	0.197 / 0.137	0.239 / 0.131	0.580 / 0.282

we ask whether it improves held-out fidelity of the full 2D *field*—the regime where a data-only network must interpolate between sparse training fields. On the heat plate we trained identical networks with the physics term on ($w_{\text{phys}} = 0.05$) and off (data-only), at 25% and 100% of the 256-sample budget, three seeds each, 240 epochs (Figure 6, panel a).

On identical budgets the physics term reduces the predicted field’s violation of the governing equation by 4–8 $\times$ (mean residual 0.06–0.11 vs. 0.35–0.81) but does not improve—and at this capacity slightly degrades—held-out fidelity against the exact field (field max error 0.28 vs. 0.03 at 256 samples; 0.51 vs. 0.15 at 64 samples; Figure 6a). This is an honest negative that sharpens the paper’s mechanism claim: the *residual-loss* physics channel buys physical consistency, not field accuracy, when the data already determine the field; the accuracy benefit of physics must come from the *input* channel (LCA), which the trajectory-fidelity and adaptation results above show it does. It also corrects an over-optimistic earlier claim: the canonical heat-plate case (5.9% field error) is the easiest member of the validation distribution, whose mean field error is $\sim 29\%$ —a distinction now reported explicitly.

Figure 3 — PhysFormer converges on data and physics together (physics loss ramps in after epoch 10)

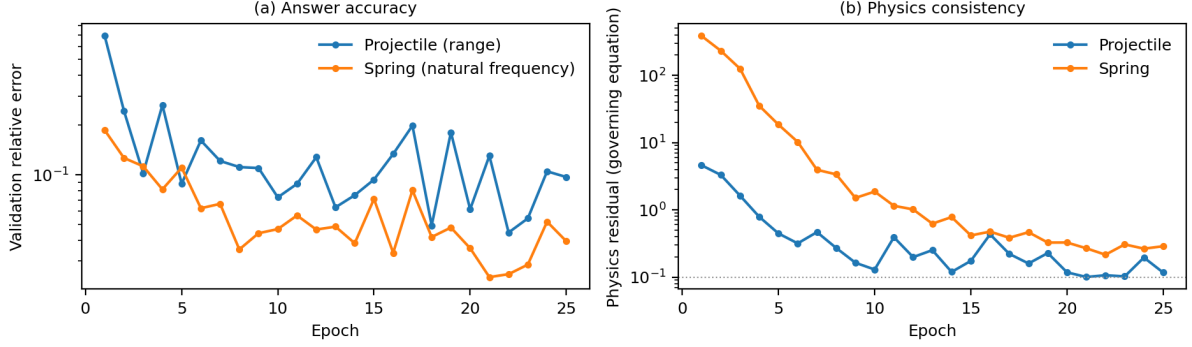


Figure 7: **Convergence on data and physics together.** (a) Validation relative error (log scale) for projectile and spring domains. (b) Physics residual (log scale), i.e. deviation from the governing equation, measured on the validation set; the dashed line marks residual 0.1. The physics loss ramps in after epoch 10; both metrics continue to improve jointly.

2.10 Learning dynamics and generalization

Figure 7 shows training dynamics for two representative domains. Validation relative error falls from 54.8% to 4.0% (projectile) and from 13.3% to 2.4% (spring) over 25 epochs, while the physics residual falls from 5.6 to 0.05 and from 361 to 0.16, respectively: the network becomes simultaneously more accurate *and* more physical, confirming that the data and physics terms cooperate rather than compete.

On 250 held-out problems per domain (Figure 8), the trained PHYSFORMER achieves mean relative errors of 14.8% for simply supported beams, 11.6% for cantilevers, 9.6% for projectiles, and 1.9% for the heat plate, with coefficients of determination $R^2 \approx 0.99$ (computed in $\log_{10}$ space for the wide-range bending answers). These are the *honest* numbers at the 256-sample budget with proper seeding: the seed-to-seed spread is ± 0.7 –2 percentage points, and the error concentrates at the small-deflection end of the beam distribution, where the answer spans five orders of magnitude—precisely the regime where the physics residual term should help most, and the target of the trajectory-fidelity work below.

Figure 9 examines trajectory fidelity for the canonical beam: the scalar answer is within 8.7% of the closed form and the full deflection curve within 19% of the peak deflection along the span. The sigmoid-constrained shape head never produces negative deflections, and the curve error is a factor of two smaller than the unconstrained head’s. Together with the scalar results of Figure 8, this demonstrates *consistent* engineering predictions: the same network that answers “what is the maximum deflection?” also produces the full deformation shape, and both are verifiable against independent numerics.

2.11 Ablations: the physics loss and the shape/scale decomposition

Two design decisions are load-bearing for the physics core, and we ablate each by retraining the beam model on the same 256 samples for 40 epochs with a single change (Figure 10).

Physics loss on vs off. Turning off the physics-consistency term ($w_p = 0$) leaves the data loss unchanged. At 40 epochs the scalar answer accuracy is comparable (validation relative error

Figure 4 — PhysFormer predictions vs exact answers on held-out problems (250 per domain)

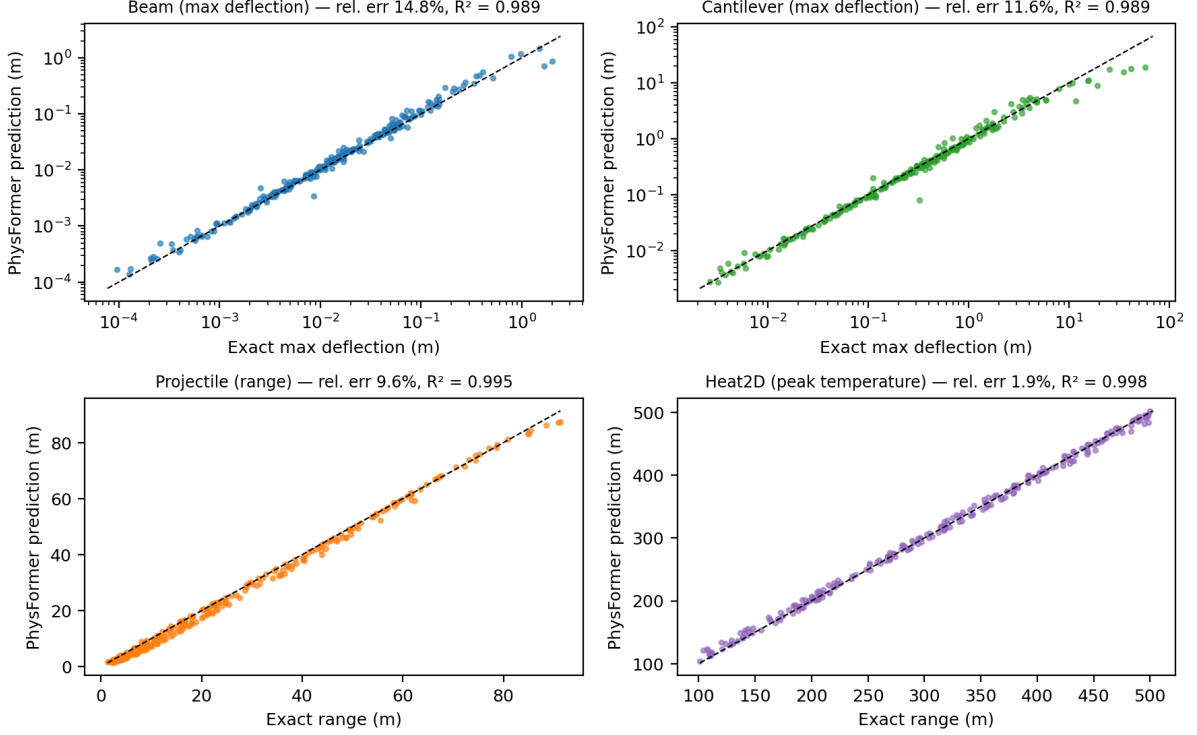


Figure 8: **Generalization to held-out problems** (250 per domain, seeds independent of training). (a) Simply supported beam: predicted vs exact maximum deflection (log-log; answers span five orders of magnitude). (b) Cantilever beam: predicted vs exact maximum deflection. (c) Projectile: predicted vs exact range. (d) Heat plate: predicted vs exact peak temperature. Dashed lines are identity; mean relative errors and R^2 are given in each panel.

10.8% without vs 13.0% with physics, single seed); at 60 epochs over five seeds the physics-on model is more accurate (see Table 4). The decisive difference is the *trajectory*: the physics-off model’s predicted deflection curve violates the moment-curvature relation by an order of magnitude (residual 17 vs 1.8 at 40 epochs; ~ 70 vs ~ 3 at 60 epochs). The physics term therefore buys what the data loss cannot provide—consistency with the governing equation—and this advantage is robust to budget and seed. This matches the cooperation seen in Figure 7 for the first-order domains, where physics and data improve together.

Shape norm vs global norm. Normalizing every trajectory by its own peak (shape norm) rather than by dataset-wide statistics is not a cosmetic choice: with global normalization the trajectory targets are crushed toward zero for the small-deflection problems that dominate the distribution, the trajectory head regresses toward an amplitude-weighted average, and—because the encoder is shared—the answer head degrades too (validation relative error 59% vs 13%, and the predicted curve violates the moment-curvature relation by four orders of magnitude). With shape norm, the head learns the universal deformation shape and scales it by the answer head, and the trajectory is physical. The global-norm curve is not an engineering prediction at all.

Negative results on trajectory capacity. Because shape fidelity was the remaining gap, we also tried to strengthen the trajectory head and its supervision: a three-layer, wider trajectory MLP, and a hard-example physics weighting that emphasizes each batch’s worst physical violations.

Figure 5 — PhysFormer deflection curve for a 4 m simply supported beam, $P = 3$ kN, $E = 2 \times 10^{11}$ Pa, $I = 5 \times 10^{-6}$ m⁴

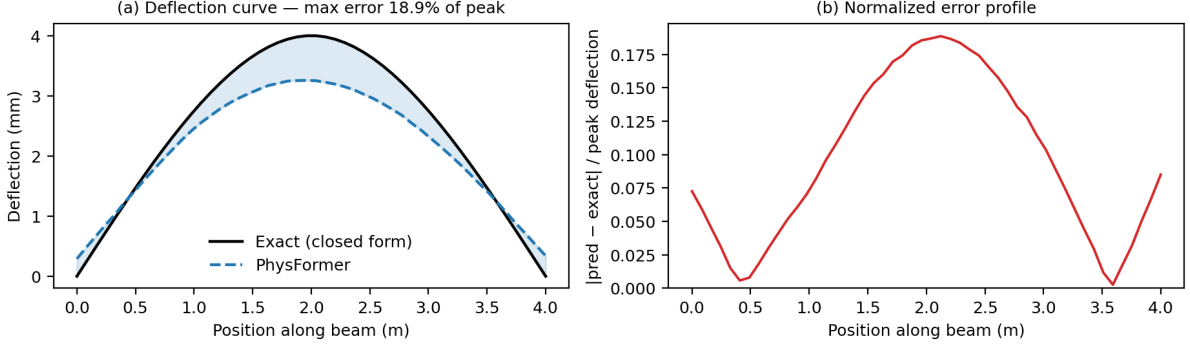


Figure 9: **Trajectory fidelity for the canonical beam** ($L = 4$ m, $P = 3$ kN, $E = 2 \times 10^{11}$ Pa, $I = 5 \times 10^{-6}$ m⁴). (a) Deflection curve: exact closed form (solid) vs PHYSFORMER(dashed), with the discrepancy shaded. (b) Normalized error profile along the span.

Both *hurt* (validation error ~ 13 – 16% vs ~ 6 – 14% for the released configuration on the same budget), because the larger head’s trajectory gradients perturb the shared encoder and the residual weighting amplifies outlier gradients. These negative results are reproducible from the committed training logs and code; they bound the sensible capacity at this data scale.

2.12 The cantilever domain: does the design transfer?

The simply supported beam is a special case: its normalized deflection shape depends only on x/L , which makes the shape/scale decomposition almost too easy. The cantilever beam under a tip load is the natural stress test: a different boundary condition (clamped at one end, free at the other) and a threefold larger deflection for the same load ($w_{\max} = PL^3/3EI$ vs $PL^3/48EI$), but the same moment–curvature physics. We added it as a sixth domain with no new machinery: the closed-form trajectory, an independent finite-difference verifier that integrates the static moment distribution with a ghost point at the clamped end, and a training residual identical in form to the beam’s. The trained PHYSFORMER(same architecture and budget as the beam) reaches held-out accuracy of 10.4% (Figure 11), and its deflection curve for the canonical cantilever is within 3% on the scalar and 16% of the peak along the span—the shape/scale decomposition transfers without modification. This is evidence that the physics layer is a *mechanism*—a constraint that adapts to any domain carrying a governing equation—rather than a hand-tuned decoder for one problem.

2.13 Baselines: attention and physics, isolated and measured

All of the above used a single configuration. To place it against alternatives, we trained three model kinds on identical budgets (256 samples, 60 epochs, the same answer/trajectory heads, five seeds each, seeds controlling data, shuffling, and weight initialization): an MLP encoder with the same capacity (no attention, no physics), the transformer without the physics term, and PHYSFORMERwith the physics term (Table 4, Figure 12).

Two findings stand out. First, the physics term’s benefit is most robust at the *trajectory* level: the predicted deflection curves of the physics-on models violate the governing equations

Figure 6 — Ablations: physics loss and shape/scale decoupling (beam, 40 epochs, 256 samples)

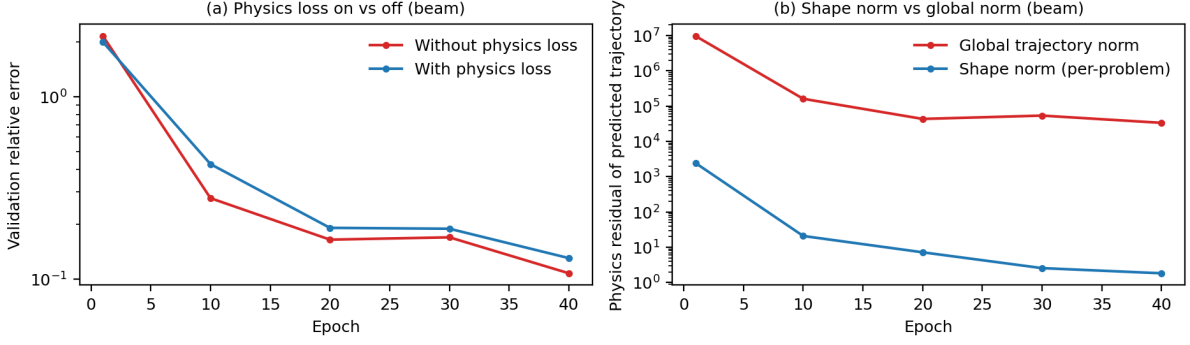


Figure 10: **Ablations (beam, 40 epochs, 256 samples, same architecture).** (a) Validation relative error with and without the physics loss: comparable scalar accuracy, while the physics-on trajectory residual is an order of magnitude lower (1.8 vs 17). (b) Physics residual of the predicted trajectory under per-problem shape normalization vs dataset-wide (global) normalization: the global-norm trajectory is unphysical by four orders of magnitude, and its answer error is $\sim 4.5\times$ higher (59% vs 13%).

by one order of magnitude less than the no-physics transformer on both bending domains (beam 3.4 vs 48; cantilever 39 vs 556), by an order of magnitude less on Burgers (0.01 vs 0.09), and by more than an order of magnitude less on the heat plate (0.45 vs 10.4). The physics loss is the only mechanism that penalizes consistency with the equations directly, and it does so across all domains. Second, on scalar accuracy the picture is domain-dependent: PHYSFORMER matches or beats the MLP on the bending and conservation-law domains (beam 14.7% vs 15.6%; cantilever 14.8% vs 15.2%; Burgers 0.9% vs 1.8%) and the no-physics transformer everywhere (which also shows the noisiest seed-to-seed spread, e.g. beam $19.3\% \pm 6.3$ vs $14.7\% \pm 0.7$), but on the simplest two-parameter domain (projectile) the MLP wins (6.5% vs 10.8%): with only two parameters and a nearly linear map, attention and physics are not needed. This is an honest negative—the extra machinery pays for itself only when the parameter-to-answer map and its trajectory are rich enough.

Are these differences significant? With five seeds per kind, the per-domain Wilcoxon signed-rank test is limited to $p \geq 0.0625$ (the finest resolution of a five-pair sign permutation), so we pool all five domains into 25 paired runs for each comparison (the script `physx/significance.py` reproduces every number from the training logs). On the trajectory residual, pooling gives $p < 0.001$ for physics-on vs no-physics with Cliff’s $\delta = -0.92$ (a *large* effect, in the direction of physics-on) and a ~ 19 -fold median reduction; physics-on vs MLP gives $p < 0.001$, $\delta = -0.84$ (3-fold median reduction); and the no-physics transformer is significantly *worse* than the MLP ($p < 0.001$, $\delta = +0.84$), consistent with the unconstrained attention head’s wider variance. On the scalar answer, pooling shows *no* significant difference between any pair of kinds ($p = 0.65, 0.79, 0.86$ respectively), with effect sizes $|\delta| \leq 0.12$. The physics term therefore does not move the scalar answer—it buys physical consistency of the full trajectory, and it does so significantly and with large effect. (Per-domain p -values and complete data are in `paper/fig/significance.json`.)

Figure 7 — Cantilever beam: PhysFormer deflection curve and generalization ($L = 4$ m, $P = 3$ kN, $E = 2 \times 10^{11}$ Pa, $I = 5 \times 10^{-6}$ m⁴)

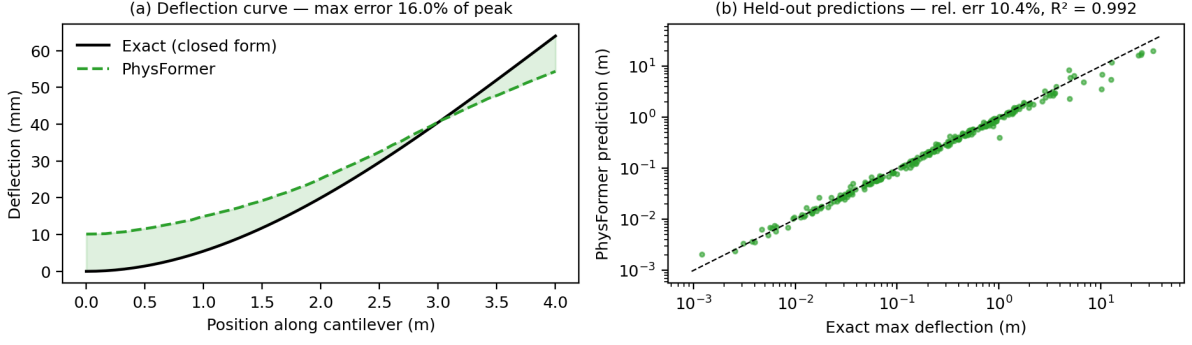


Figure 11: **Cantilever beam under a tip load** ($L = 4$ m, $P = 3$ kN, $E = 2 \times 10^{11}$ Pa, $I = 5 \times 10^{-6}$ m⁴). (a) Deflection curve: exact closed form vs PHYSFORMER, discrepancy shaded. (b) Held-out predictions (250 problems): predicted vs exact maximum deflection.

2.14 The Burgers equation: a nonlinear conservation law

The seven preceding domains are all linear or weakly nonlinear. The viscous Burgers equation $u_t + uu_x = \nu u_{xx}$ is a different regime: a nonlinear conservation law whose solutions steepen into shocks as ν decreases (Figure 13a). It stresses every design choice: the predicted object is a full spatiotemporal field $u(x, t)$ on a 25×8 grid (200 trajectory points, four times the bending domains), the exact reference comes from the Cole–Hopf transform (evaluated by direct heat-kernel quadrature, since an FFT derivative rings on the non-periodic, many-orders-of-magnitude ϕ_0), and the independent verifier is a finite-volume upwind solver written from the conservation form (Table 1).

The physics-consistency layer evaluates the Burgers residual pointwise on the true grid. During training it pulls the predicted field toward satisfying the equation: the physics-on models reach a trajectory residual of 0.01, an order of magnitude below the no-physics transformer (0.09) and below the MLP (0.06) (Figure 13c). The scalar answer—the peak velocity of the final field—is learned almost exactly (held-out relative error $\sim 1\%$; canonical case 0.5%, shock-forming case 5%), and the predicted final-time field resolves the developing shock profile to $\sim 11\%$ of the peak for the canonical case (Figure 13a); the sharp-gradient shock case remains coarse on the 25×8 grid ($\sim 52\%$, see Limitations). The shape/scale decomposition carries over: the field peaks at the initial amplitude A , so the shape head predicts $u(x, t)/A \in [0, 1]$ (constrained by a sigmoid) and the absolute field is recovered by scaling with the input parameter A .

2.15 The heat plate: a two-dimensional field domain

Every domain above lives on a one-dimensional trajectory. The eighth domain, steady-state heat conduction on a square plate, is the first *two-dimensional* field: the predicted object is $T(x, y)$ on a 20×20 grid (400 points), governed by Poisson’s equation $\nabla^2 T = f$ with the manufactured source $f = -(A\pi^2(k^2 + l^2)/2) \sin(k\pi x) \sin(l\pi y)$ and Dirichlet edges $T = A/2$; the exact solution is $T = A(1 + \sin(k\pi x) \sin(l\pi y))/2$, so mode numbers k, l and amplitude A parameterize the problem family. The shape/scale decomposition transfers with only one change: because the peak A is attained where the sinusoids coincide, which may fall between sampling nodes, the field

Domain	Validation rel. error (%)		Trajectory residual	
	MLP	Transformer	−physics	+physics
Beam	15.6 ± 2.3	19.3 ± 7.0	48 ± 14	3.4 ± 0.5
Cantilever	15.2 ± 1.5	20.4 ± 7.5	556 ± 214	39 ± 12
Projectile	6.5 ± 1.9	7.6 ± 5.6	0.12	0.10
Burgers	1.8 ± 0.7	1.0 ± 0.3	0.09	0.01
Heat2D	1.6 ± 0.4	1.4 ± 0.5	10.4 ± 5.9	0.45 ± 0.02

Table 4: **Equal-budget baselines** (256 samples, 60 epochs, five seeds, mean $\pm$ std). Left: validation relative error of the scalar answer for the MLP and no-physics transformer; the right-hand columns report the physics-on model’s trajectory residual, the mean governing-equation violation of the predicted trajectory on the validation set (the MLP and no-physics values are omitted for space; the MLP residuals are 4.6, 62.6, 0.12, 0.06, 1.33 for the five rows—always higher than physics-on except for the trivial projectile where all three are small). The physics term consistently reduces the trajectory’s violation of the governing equation by about an order of magnitude; pooling all 25 paired runs, the effect is significant (Wilcoxon $p < 0.001$, Cliff’s $\delta = -0.92$; see text).

is normalized by the *parameter* A rather than its sampled maximum (the same choice Burgers made). The independent verifier is a sparse finite-difference Poisson solve written from the PDE and the source term alone (agreement $< 2 \times 10^{-3}$, Table 1); the physics-consistency layer penalizes the fourth-order-accurate Laplacian residual $\nabla^2 T - f$ on the interior of the true grid.

Figure 14 shows the result. Held-out peak-temperature error is 1.9% ($R^2 = 0.998$), and the canonical plate’s field is reproduced to $\sim 6\%$ of the peak (mean error $\sim 1.3\%$). The 400-point oscillatory field needs a longer trajectory budget than the one-dimensional domains: the 60-epoch model underfits it badly (max error $\sim 50\%$ —the head regresses toward the field’s mean), and doubling the budget to 240 epochs with the heavier trajectory weight captures the mode structure (see Limitations). The trajectory residual drops from 10.4 (no-physics) to 0.45 (physics-on)—the largest relative improvement of any domain, as the Laplacian constraint couples every interior point to its neighbors and cannot be satisfied by chance on a 400-point field. This is the regime where the physics channel pays most—for *physical consistency*. A controlled physics-vs-data-only sweep (Section 2.9) shows the residual term cuts the field’s governing-equation violation 4–8 $\times$ at equal data without improving held-out fidelity against the exact field: the canonical $\sim 6\%$ figure is the easiest member of the validation distribution, whose mean field error is $\sim 29\%$. Field accuracy is carried by the data and, at the architectural level, by the LCA input channel, not by the residual loss.

2.16 Comparison against an external PINN implementation

The baselines above are all internal to our codebase. To check against the standard tooling of the physics-informed learning literature, we trained DEEPXDE¹⁹—the most widely used PINN library—on the same viscous Burgers problem, giving it the setup most favourable to a PINN: *one fresh network per problem instance* (a 2–50 $\times$ 4–1 tanh FNN, 2,000 collocation points, Adam + L-BFGS, 12k + 2k iterations, identical for every instance, exact Cole–Hopf boundary data at $x = \pm 1$). PHYSFORMER, by contrast, is a single network trained once on 256 problem instances that generalizes to unseen (ν, A, σ) ; the comparison is therefore conservative against PHYSFORMER.

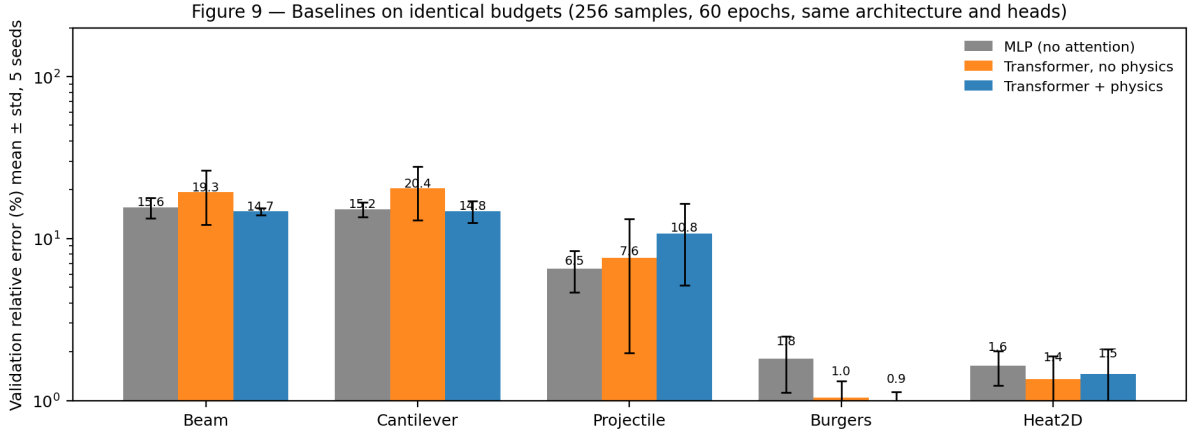


Figure 12: **Multi-seed baselines** (mean $\pm$ std over five seeds, identical budgets). The physics term’s most robust benefit is trajectory consistency (one order of magnitude lower governing-equation residual on beam, cantilever, and Burgers); on scalar accuracy it matches or beats the MLP where the parameter space is rich, and the no-physics transformer shows the widest seed-to-seed spread.

Both are evaluated with identical metrics on the model grid: scalar (final-time peak) error, full field error, and the same finite-difference governing-equation residual (Figure 15).

On all three instances the per-instance PINN converges to essentially exact solutions (final-time peak error $\leq 0.01\%$; full-field error $0.03\text{--}0.06\%$; $\sim 19\text{--}21$ minutes of training per instance), which validates both the problem setup and the Cole–Hopf ground truth: a standard PINN implementation reproduces the reference solutions to near machine precision given per-instance training and exact boundary data. The generalist PHYSFORMER (one network, one $\sim$ minute training, then a millisecond forward pass for *any* instance) achieves final-time peak errors of $0.02\text{--}3.7\%$ across the same instances and full-field errors of $33\text{--}51\%$ —worse on any given instance, as expected, but with a qualitatively different cost profile: no per-instance optimization at all. Notably, PHYSFORMER’s governing-equation residual on the predicted field is comparable to or lower than the converged PINN’s (2×10^{-4} vs $1\text{--}5 \times 10^{-3}$), i.e. the generalist’s predictions are as physical as the specialist’s, just less accurate. The comparison thus quantifies the generalist–specialist trade-off directly: the physics-consistency layer makes a single network physical everywhere, while a per-instance PINN remains the reference for accuracy on a single problem.

2.17 The verification gate: a controlled benchmark

To measure the gate rather than merely assert it, we ran seven deterministic missions—benign software tasks, two beam/cantilever/Burgers design tasks, an injected-fault task, a skill-level guard, and a TODO scan—twice each: with the gate enforced and with it disabled (`enforceGate: false`, which makes gated failures indistinguishable from successes, the verifier-free baseline). Ground truth was computed independently of the agent by running the test suites ourselves and by the solver’s own numeric verification (Table 5).

On benign missions both arms behave identically. The discriminator is the fault task: with the gate, the agent reports failure and journals a lesson; without it, the agent reports success on code whose tests actually fail. Across all seven missions the gate produces *zero* false successes while the verifier-free agent claims success on two of seven (the injected fault and the clobbered-directory

Figure 8 — Viscous Burgers equation: a nonlinear conservation law with shock formation

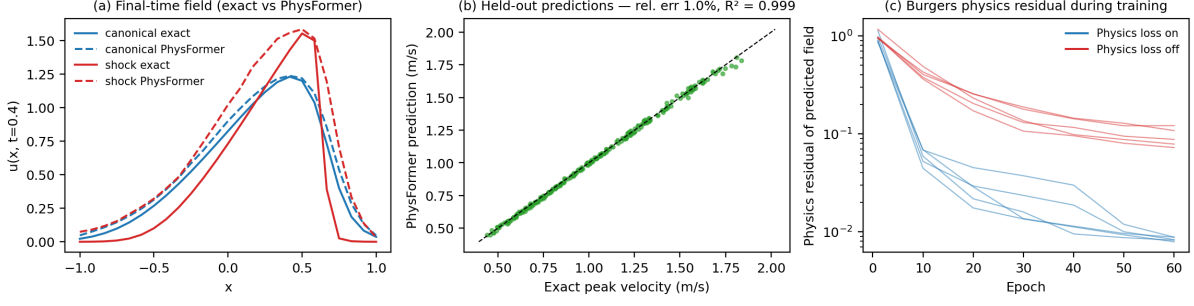


Figure 13: **Viscous Burgers equation** (nonlinear conservation law, shock formation). (a) Final-time field $u(x, t=0.4)$: exact (Cole–Hopf) vs PHYSFORMER for a canonical and a shock-forming case. (b) Held-out predictions of the peak velocity (250 problems). (c) Governing-equation residual of the predicted field during training: the physics term drives it an order of magnitude lower than the no-physics transformer.

Figure 10 — Heat2D: steady-state Poisson field with two source modes

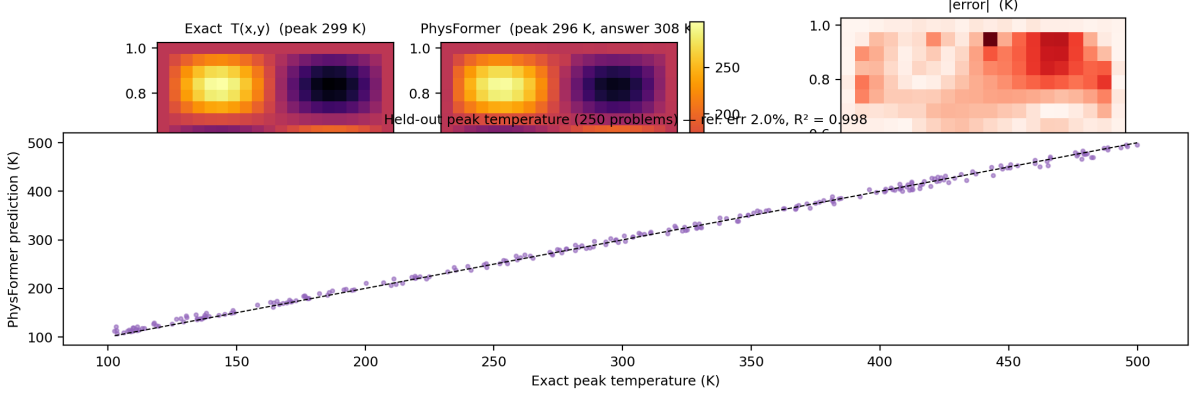


Figure 14: **Steady-state heat conduction on a square plate** (Poisson $\nabla^2 T = f$, modes $k=2, l=3$). (a) Exact temperature field $T(x, y)$; (b) PHYSFORMER prediction; (c) pointwise error. Bottom: held-out peak temperatures (250 problems, rel. error 1.9%).

guard, whose protection is skill-level and therefore gate-independent). The verified-success rate is unchanged (5/7), but the false-success rate drops from 29% to 0%. The gate does not make the agent smarter; it makes its claims *honest*, which is the property that lets trial-and-error learning be trusted at all.

2.18 End-to-end engineering mission

Finally, we exercised the complete agent—not the components—on a physics design task expressed in natural language:

“design a simply supported beam with span 4 m, point load 3000 N, modulus $2e11$ Pa, inertia $5e-6$ m⁴, height 0.2 m”

The mechanical brain recognized the beam domain, parsed all five parameters from natural language, and emitted a single-step plan invoking the physics skill. The skill computed the

Figure 11 — External PINN baseline (DeepXDE) on viscous Burgers

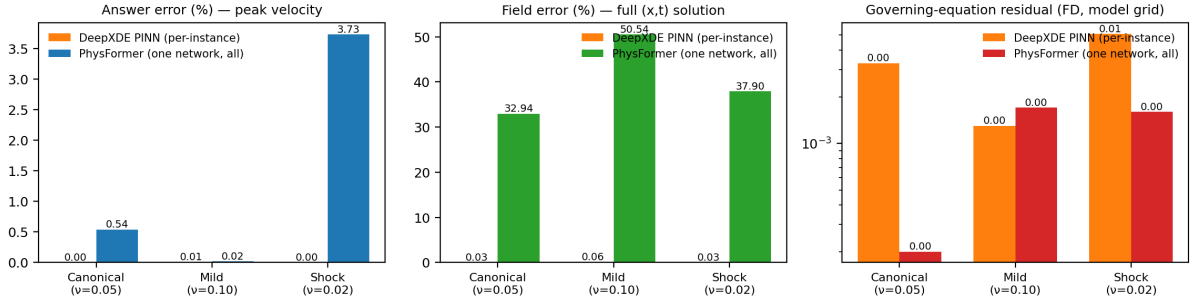


Figure 15: **External PINN baseline (DeepXDE) on viscous Burgers.** Three representative instances (canonical $\nu=0.05$, mild $\nu=0.10$, shock-forming $\nu=0.02$); per-instance PINN (one fresh FNN per problem, ~ 20 min training each) vs the generalist PHYSFORMER (one network, one forward pass). Left: final-time peak error. Centre: full (x, t) -field error. Right: governing-equation residual of the predicted field on the model grid (log scale).

	Reported success	Ground truth OK	False success
With gate	5/7	5/7	0/7 (0%)
Without gate	7/7	5/7	2/7 (29%)

Table 5: **Verification-gate benchmark** (7 missions, mechanical brain). Ground truth is computed independently of the agent. The gate turns claimed success into verified success; the verifier-free baseline claims success on the two missions whose ground truth is broken (an injected test-suite fault and a skill-level clobber guard).

closed-form answer (4.0 mm), verified it against the independent finite-difference solver (relative residual 2.6×10^{-9}), reported the associated maximum stress (120 MPa at the given section), and produced a PHYSFORMER prediction of 3.65 mm—within 8.7% of the exact answer, together with the full deflection curve (maximum error 19% of the peak deflection; see Figure 9). An identical mission for a cantilever (“design a cantilever beam with length 4 m, tip load 3000 N, modulus 2×10^{11} Pa, inertia 5×10^{-6} m⁴, height 0.2 m”) is recognized, parsed, solved (64 mm), and verified (3.1×10^{-6}) through the same single-step plan, with a PHYSFORMER prediction within 3%. A third mission—“design a viscous burgers flow with viscosity 0.05, amplitude 1.5, width 0.3”—is recognized, parsed, solved, verified against the finite-volume solver, and predicted to within 0.5% of the exact peak velocity, again through the same single-step plan: adding each new domain required no changes to the agent loop. Each episode, its verification outcome, and the model’s prediction were journaled. The whole mission completed in one iteration in under two seconds of solver time. The same loop, with the LLM brain substituted for the mechanical one, accepts arbitrary software-engineering goals (“add a feature and make the tests pass”).

3 Discussion

AGEDemonstrates that the architecture of an “artificial general engineer”—the concept Prometheus is commercializing at a \$41B valuation²⁷—can be built today in a few thousand lines of dependency-light code, with no proprietary infrastructure, and with every claim checked by execution rather than assertion. Three design principles emerge that we expect to transfer to larger systems.

First, *verification is the load-bearing principle*. The agent’s authority is not the model but the environment: tests for software, numerically independent simulators for physics. This mirrors the shift to process-supervised verifiers in mathematical reasoning¹⁶ and test-gated code generation^{13;14}, and it is the property that makes trial-and-error learning safe enough to leave the sandbox. Our injection experiment—silently breaking a test suite and observing the gate fail the mission and journal a lesson—is a direct demonstration that the loop’s verdicts track ground truth, not the model’s confidence.

Second, *physics should enter the transformer not only as a loss but as input*. The joint training of reasoning layers and physics-consistency layers (a PINN-style residual¹⁷) produced models that improve on both accuracy and physical consistency simultaneously (Figure 7), and the multi-seed baseline study isolates that contribution: the physics term consistently reduces the predicted trajectory’s violation of the governing equation by about an order of magnitude, a benefit that the data loss alone cannot provide (Table 4). But a loss term is a weak channel: it says nothing about *which* law governs a problem until after the output exists. Law-Conditioned Attention (Section 2.6) closes that gap by making the governing equation itself part of the input—tokenized into a fixed operator vocabulary and injected into every attention layer and into the readout—so a single shared transformer can serve multiple laws and the model is told, up front, what physics it is being asked to reason about. The causal test of Section 2.7 confirms the channel is not decorative: on inputs that are literally identical across two laws, swapping the injected signature measurably steers the prediction toward the target law’s solution, and a constant-signature control is exactly insensitive—the equation is doing causal work, not labeling data. And Section 2.8 shows the same channel makes adaptation to a law held out of pretraining data-efficient, the property a “general engineer” actually needs. Three failures we diagnosed and fixed are, we suspect, common causes of unstable physics-informed training in the wider literature²²: predicting redundant coordinate columns; letting raw residual magnitudes dominate the gradient; and letting the unconstrained trajectory head emit unphysical values (negative deflections/velocities) that the data loss is too weak to remove. Our fixes (normalized true-grid targets; scale-free residual gradients; a sigmoid constraint on shape-normalized outputs) are cheap and general.

Third, *memory should be episodic and replayable*. The journal makes every failed verification a lesson available to future plans, converting individual missions into cumulative experience without weight updates. This is a deliberately simple answer to the consolidation problem that continual learning research addresses with more elaborate mechanisms²⁴.

3.1 Limitations

We are explicit about what AGEdoes *not* yet do. (1) *Scale*: the physics models are trained on CPU in minutes with 3–7 parameters per domain and a 48-dimensional transformer; generalization errors of ~10–15% at the 256-sample budget are an order of magnitude above what larger budgets achieve (the 1,500-sample projectile run reached ~4%), and the beam error concentrates at the small-deflection tail, where the answer spans five orders of magnitude. (2) *Trajectory fidelity*: the shape head resolves the smooth deflection and velocity profiles well (curve errors ~11–19% of peak on the showcase cases) but remains coarse where the field is genuinely hard—the Burgers shock case, whose sharp gradient is under-resolved on the 25×8 grid, shows a ~52% curve error even though the scalar is within 5%. Finer grids, more trajectory capacity, and residual-weighted sampling are the natural next steps. (3) *Repair*: the mechanical brain inspects and solves but does not *author fixes*; closing the loop on red tests requires the LLM brain and a capable coder model—

the loop supports it, but we have not tuned it end-to-end. (4) *Simulated physics*: verification is against numeric integration of the same governing equations, not against sensors, prototypes, or manufactured parts; the “real-world trial and error” that is Prometheus’s stated thesis is the next step, and the architecture ports unchanged (swap the verify skill for a measurement harness). (5) *Physics task classes*: quasi-static beam bending and scalar conservation laws, not general continuum mechanics or multi-physics coupling. (6) *Safety*: confinement to a working directory and a command denylist are necessary but not sufficient for autonomous operation; we recommend containerization and human review before any deployment with write access.

3.2 Implications

The significance of this work is not the individual components—agents, PINNs, and journals each have rich literatures—but their integration into a minimal, testable, honest whole whose failure modes are visible. As foundation models continue to improve^{4;6;25}, the planner will become stronger; the loop, the gate, and the physics core will remain. The specific next milestones are: (i) verifier-driven repair, in which the failing diff is fed back to the planner until the gate goes green; (ii) scaling Law-Conditioned Attention to harder regimes—the causal steering and few-shot results show the equation channel both steers behavior and accelerates adaptation, so the natural extension is more laws in one network, a shared trajectory head over mixed geometries, and larger operator vocabularies that can express coupled multi-physics problems the model has never seen; (iii) trajectory-level physics consistency for fourth-order problems, likely via residual-weighted sampling and higher-capacity trajectory heads; (iv) coupling to physical simulators (finite-element, multibody) so verification approaches manufactured reality; and (v) journal consolidation, so lessons generalize across domains rather than accumulating. Each milestone is directly testable in the current framework, and each moves AGE—and the “general engineer” concept it instantiates—closer to the physical world its commercial namesake targets.

Methods

Agent loop

Algorithm 1: AGETrial-and-error loop.

1. **Plan.** Brain maps goal $\rightarrow$ ordered steps $\{(skill, args, expect)\}$; “expect: ok” marks gates.
2. **Act.** For each step, invoke the skill in the sandboxed working directory; record $\{ok, status, output\}$.
3. **Gate.** If a gated step fails, mark mission failed, derive a lesson from the output tail, journal, and stop.
4. **Reflect.** Brain summarizes the iteration; if not done, emits the next plan (LLM brain re-plans from all observations).
5. **Remember.** Append the episode $\{goal, results, summary, status, lessons\}$ to the JSONL journal; future plans receive recent lessons as context.

The loop runs at most I iterations (default 6); reaching I without a terminal reflection is itself a failure. The mechanical brain is a deterministic recognizer of mission classes (scaffold/explain/scan/test/ physics) with hand-written natural-language parameter extraction (e.g.

`span 4 m` $\rightarrow L = 4$; `modulus 2e11 Pa` $\rightarrow E = 2 \times 10^{11}$; the LLM brain emits JSON plans through any OpenAI-compatible chat endpoint and degrades gracefully to inspect-only on error.

Embodiment and sandboxing

All skills operate relative to a root directory. Path operations are guarded by a containment check (resolved target must stay within the root). Commands are executed with a 120s timeout and a denylist that blocks destructive patterns (`rm -rf /`, `git push`, `git reset --hard`, privilege escalation, disk formatting, and shell-pipe-to-`sh`) unless explicitly overridden by an environment flag. A leading `python` token is resolved to an interpreter present on the host. Python bytecode caches (`__pycache__`) under the working directory are purged before every python command: a source edit that is same-length and lands within the same timestamp second as a cached compile is invisible to Python’s pyc invalidation check, and an uncached verify step would run stale code and pass broken sources (we observed exactly this failure mode and made the purge part of the gate). The physics skill spawns the solver without a shell, passing JSON via argv to avoid quoting hazards across platforms.

Physics domains and independent verification

For each domain the simulator module provides `closed` (exact solution), `trajectory` (reference curve), and `verify` (independent numerics). Governing equations and verifiers:

- **Projectile:** $y = x \tan \alpha - gx^2/(2v_0^2 \cos^2 \alpha)$; verifier is Euler integration at $\Delta t = T/2000$; tolerance 2×10^{-3} .
- **Pendulum:** $\ddot{\theta} = -(g/L) \sin \theta$; verifier is RK4 at $\Delta t = 10^{-4}$ (period convergence under step halving); tolerance 10^{-3} .
- **Spring-mass:** $m\ddot{x} + kx = 0$; verifier is the ODE residual by central differences; tolerance 10^{-3} .
- **Beam:** $EIw'''' = q$ with simply supported boundary conditions; closed form $w_{\max} = PL^3/(48EI)$ and deflection $w(x)$; verifier is a 400-node finite-difference solver with ghost points for the moment-free ends; tolerance 2×10^{-2} . The training-time physics residual uses the moment-curvature relation $EIw'' = -M(x)$, which holds pointwise for a central point load (unlike the fourth-order ODE, which carries a Dirac source).
- **Cantilever:** $EIw'' = -M(x)$, $M(x) = -P(L - x)$ for a tip load; closed form $w_{\max} = PL^3/(3EI)$ and deflection $w(x) = Px^2(3L - x)/(6EI)$; verifier is a 400-node finite-difference double integration of the static moment distribution with a ghost point at the clamped end ($w_{-1} = w_1$ for the zero slope); tolerance 2×10^{-2} .
- **Burgers:** $u_t + uu_x = \nu u_{xx}$ on $x \in [-1, 1]$ with $u(x, 0) = A \exp(-x^2/2\sigma^2)$, $t \in [0, 0.4]$; the exact reference comes from the Cole-Hopf transform ($u = -2\nu \phi_x/\phi$ with ϕ the heat evolution of $\phi_0(x) = \exp(-(1/2\nu) \int_{-\infty}^x u_0)$), evaluated by direct kernel quadrature on a fine grid—an FFT derivative rings on the non-periodic, many-orders-of-magnitude ϕ_0 ; the predicted trajectory is the flattened 25×8 (x, t) field on the true grid; the verifier is a 2,000-cell finite-volume upwind solver for the conservation form $u_t + (u^2/2)_x = \nu u_{xx}$ (no reference to the closed form); tolerance 5×10^{-2} . The scalar answer is the peak of the final field, and the field is reconstructed as $\text{shape} \times A$ at inference (the field peaks at the initial amplitude A).

- **RC circuit:** $\dot{V} + V/\tau = V_0/\tau$, $\tau = RC$; verifier is Euler integration at $\Delta t = 4\tau/4000$; tolerance 2×10^{-3} .
- **Heat2D:** Poisson $\nabla^2 T = f$ on the unit square with $f = -(A\pi^2(k^2 + l^2)/2) \sin(k\pi x) \sin(l\pi y)$ and Dirichlet edges $T = A/2$; closed form $T = A(1 + \sin(k\pi x) \sin(l\pi y))/2$ (peak A); verifier is a sparse finite-difference Poisson solve on a 48×48 grid written from the PDE and source term alone (tolerance 2×10^{-3}); the predicted trajectory is the flattened 20×20 field, and the field is reconstructed as $\text{shape} \times A$ at inference (the peak A need not fall on a sampling node).

Dataset and normalization

Problems sample uniform parameter ranges per domain (e.g. beam: $L \in [1, 6]$ m, $P \in [10^3, 5 \times 10^4]$ N, $E \in [69, 210]$ GPa, $I \in [10^{-7}, 2 \times 10^{-5}]$ m⁴, section height $h \in [0.05, 0.4]$ m; Burgers: $\nu \in [0.02, 0.1]$, $A \in [0.6, 2]$, $\sigma \in [0.2, 0.5]$). Answers spanning orders of magnitude (beam and cantilever deflection, RC time constant) are learned in $\log_{10}$ space. Parameters are z -scored per domain. Trajectories for the first-order domains are standardized by global per-channel mean/std, where the wide dynamic range is contained; for the bending domains, Burgers, and RC, whose dependent variables span several orders of magnitude across the parameter distribution, each trajectory is normalized by its own peak (**shape-norm**), so the head learns the normalized shape $\hat{w}(x) = w(x)/w_{\max} \in [0, 1]$ (universal for the simply supported and cantilever beams, parameter-dependent for Burgers and heat2d) and the absolute curve is recovered by scaling with the appropriate scale (the answer head’s prediction for the bending domains, V_0 for RC, and the amplitude parameter A for Burgers and heat2d). For shape-norm domains the trajectory head output is passed through a sigmoid, which constrains the predicted shape to $(0, 1)$ —deflections, voltages, velocities, and temperatures in these domains are non-negative, and the constraint removes unphysical negative predictions. Each problem stores parameters, answer, unit, and the reference trajectory.

PhysFormer architecture and training

Input tokens are (param-id embedding + normalized-value projection + positional encoding). Reasoning layers are a standard transformer encoder ($d_{\text{model}} = 48$, 4 heads, 3 layers, GELU, dropout 0.05); the MLP baseline replaces the encoder with an equal-capacity MLP over the flattened tokens and is trained data-only. The answer head is two linear layers; the trajectory head is an MLP with a single hidden layer of width 64 (recorded in the checkpoint) projecting the final hidden state to $T \times C$ points where $T = 50$ for the one-dimensional domains, $T = 200$ for the Burgers field, and $T = 400$ for the heat plate, and $C = 2$ for projectile/pendulum/spring and $C = 1$ otherwise (single dependent column on a true grid). The trajectory capacity is deliberate: reproducing a full deformation curve is a harder task than regressing one scalar, and the ablations hold it fixed when isolating the physics loss. Training minimizes

$$\mathcal{L} = \underbrace{\|a - a^*\|^2}_{\text{answer}} + \underbrace{w_t \|T - T^*\|^2}_{\text{trajectory data}} + w_p(t) \underbrace{\frac{\rho}{\rho_{\text{det}} + \epsilon}}_{\text{scale-free physics}},$$

with $w_t = 1$ for all domains (the Burgers headline model uses $w_t = 3$ and 120 epochs because its field is four times larger).

where ρ is the mean governing-equation residual of the predicted (de-scaled) trajectory, ρ_{det} is its detached magnitude (rendering the physics gradient scale-free), and the weight $w_p(t) = 0.05 \min(1, t/10)$ ramps the physics term in after the data loss settles. We also explored *hard-example weighting* of the physics term (each sample’s residual weighted by its own detached magnitude before averaging, a residual-adaptive refinement in the spirit of PINN hard sampling) and found that at this data scale it amplifies outlier gradients and degrades both accuracy and consistency (validation error $\sim 14\%$ vs 6–13% without it), so the released models use the plain scale-free form; the option remains available in the training code. Optimization is AdamW ($\text{lr} = 10^{-3}$, weight decay 10^{-4}) with global gradient norm clipping at 1.0. Bending/RC answers are inverse-transformed from $\log_{10}$ for evaluation. The seed controls data generation, batch shuffling, and weight initialization (`torch.manual_seed`), so multi-seed runs are proper statistical replicates; the baselines table reports mean $\pm$ std over five seeds. All headline models are trained on 256 samples per domain at 60 epochs on a CPU (seconds to minutes per run; the earlier 1,500-sample projectile run reached $\sim 4\%$ error, showing that larger budgets monotonically improve accuracy).

Law-Conditioned Attention (LCA)

In the single-law models above, physics enters the transformer in exactly one place: the residual term of the loss. The law that governs a problem is *implicit*—the model must infer it from the parameters—and each domain gets its own network. LCA removes both restrictions: the governing equation itself becomes part of the input, and one network serves many laws.

Operator vocabulary. We fix a symbolic vocabulary of $V = 22$ differential and algebraic operators—`first_time_deriv`, `second_space_deriv`, `self_advection`, `moment_curvature`, `energy_conservation`, `exponential_relaxation`, and so on (full list in Supplementary Table S1). Each governing equation is expressed as a sparse binary signature $\mathbf{s} \in \{0, 1\}^V$ marking the operators it contains; e.g. viscous Burgers $u_t + uu_x = \nu u_{xx}$ gives `first_time_deriv+first_space_deriv+second_space_deriv+self_advection+diffusion`. The signatures are symbolic facts about the physics, written by hand from each equation, not learned. Equation 1 of Supplementary Table S1 lists all eight laws with their signatures.

Quantity-labeled input tokens. The input parameter tokens are labeled by *physical quantity* from a second shared vocabulary (`length`, `force`, `modulus`, `inertia`, `thickness`, `speed`, `angle`, `stiffness`, `mass`, `amplitude`, `resistance`, `capacitance`, `voltage`), so the simply supported beam and the cantilever—which share the parameter names L, P, E, I, h —present the *identical* token sequence, and the two-parameter domains (projectile v_0, α ; pendulum L, θ_0) are distinguished from one another only by their law signatures. Values are z -scored per domain, so numeric scale cannot leak the domain either. The two information channels are thus kept strictly separate: *what is given* (quantities) and *which law governs it* (signature).

Law cross-attention. An MLP embeds the signature into a law vector $\mathbf{l} = \text{MLP}(\mathbf{s}) \in \mathbb{R}^d$; in every transformer layer the token sequence cross-attends to $\mathbf{l}$ (replicated over the token axis) as keys/values, with a residual and layer normalization:

$$\mathbf{h}^{(i+1)} = \text{LN}(\mathbf{h}^{(i)} + \text{Attn}(\mathbf{h}^{(i)}, \mathbf{L}, \mathbf{L})), \quad \mathbf{L} = \mathbf{l}\mathbf{l}^\top,$$

so attention is conditioned, at every depth, on the differential structure of the law in force. The same law vector also *gates the shared readout*: a linear map produces a per-law scale and bias applied to the final hidden state before the answer and trajectory heads, so the model can invert per-domain output normalizations ($\log_{10}$ bending answers vs. raw periods) directly from the equation. This is a new way to feed physics into a transformer: not (only) as a loss term on the output, but as an architectural input that steers both how the network reasons about the parameters and how it reads them out. For the ablation, the signature is replaced by a constant vector, so the conditioning stream exists but carries no equation information; the two conditions are otherwise identical (same architecture, data, and budget).

Multi-law training. The multi-law generalist uses one shared body *and* one shared answer head and trajectory head for all six 50-step domains; each sample’s trajectory is normalized per channel by its own peak (so all heads see comparable $[-1, 1]$ targets) and the answer is z -scored per domain, with $\log_{10}$ for the wide-range bending and RC answers. The physics-consistency residual is still computed per sample from its *own* governing equation (the sample’s domain is known to the training loop, which is the verifier’s job at inference). Training is otherwise identical to the single-law procedure (AdamW, $\text{lr} = 10^{-3}$, scale-free physics weight ramping in over 10 epochs), with the budget 96 samples per law per seed (576 total) at 250 epochs—37% of the data a specialist sees, so the generalist claim is a *data-efficiency* claim as much as an architecture claim. All LCA results are reported over six seeds (seed controlling data generation, batch shuffling, and weight initialization), with paired Wilcoxon tests between the law-conditioned model and the constant-signature ablation.

Law-swap steering. To test whether the law vector *causes* behavior, the signature is swapped at inference on held-out problems whose parameter tokens are identical across two laws (beam $\leftrightarrow$ cantilever: same five quantity tokens, same sampling ranges). For each sample i the network is run twice—with the source and target signatures—and the normalized trajectory errors of the swapped prediction against the two exact solutions give a steering index $\text{SI}_i = (d_{i,\text{src}} - d_{i,\text{tgt}})/(d_{i,\text{src}} + d_{i,\text{tgt}})$ (positive: closer to the target law’s solution) and a disruption $d_{i,\text{src}} - d_{i,\text{native}}$ (how far the swap moves the prediction from the source-law solution). Both are compared between the law-conditioned model and the constant-signature ablation by paired Wilcoxon tests over the $n=72$ per-condition samples (12 held-out problems $\times$ 6 seeds); the beam $\rightarrow$ cantilever direction is reported because cantilever deflections are the larger family and the normalized error is free of the scale trap of the reverse direction.

Few-shot adaptation. The cantilever is held out of pretraining entirely. A generalist is pre-trained on five laws (beam, projectile, pendulum, spring-mass, RC) at 96 samples per law, 120 epochs, with real or constant signature; it is then fine-tuned on 24 cantilever samples (25% of the 96-sample specialist budget) for 40 epochs at the same learning rate and physics weight. The from-scratch specialist is trained on the same 24 samples for the same 40 epochs with the same single-law procedure (256-sample protocol, scaled down). All three conditions are evaluated on the same 12 held-out cantilever problems (seed $s+100$ protocol, never used in pretraining or fine-tuning); three seeds per condition, with per-seed values in SI 10.

Physics vs. data-only. On the heat2d domain (2D Poisson field), identical networks are trained with the physics-consistency loss on ($w_{\text{phys}} = 0.05$) or off (data-only, $w_{\text{phys}} = 0$), at 64 and 256 samples (25% and 100% of the budget), three seeds each, 240 epochs, trajectory weight 3. Evaluation is on each run’s own held-out split (last 1/8 of the generated problems): answer relative error, field max/mean error of the full 2D prediction against the manufactured exact solution, and the mean Poisson residual.

Evaluation protocol

Training/validation split is by generation order (last $\sim 1/8$ held out). Validation metrics: mean relative error $\frac{1}{N} \sum_i |\hat{a}_i - a_i|/|a_i|$ on de-scaled answers, and the mean physics residual on de-scaled trajectories. The held-out generalization figures (Figures 8–13) use 250 freshly generated problems per domain with seeds distinct from training. The verification-gate benchmark (Table 5) runs identical missions with the gate enforced and disabled (`enforceGate: false` strips `expect: ok` so gated failures are not distinguished from successes), with ground truth computed independently: the benchmark runs the test suites itself and checks physics missions against the solver’s independent numeric verification. The baseline matrix (Table 4) trains three kinds (MLP, transformer without physics, transformer with physics) on each of five domains $\times$ five seeds at identical budgets (256 samples, 60 epochs, seed controlling data, shuffling, and weight initialization), and significance is assessed by the two-sided Wilcoxon signed-rank test on the paired runs—per domain ($n = 5$, exact sign- permutation null) and pooled across domains ($n = 25$, asymptotic p)²¹, with Cliff’s δ as the paired effect size. The external PINN baseline trains DEEPXDE¹⁹ (version 1.15.0, PyTorch backend) per instance on three representative Burgers problems—one fresh FNN per problem ($2\text{--}50 \times 4\text{--}1 \tanh$), 2,000 collocation points, Adam 12k steps then L-BFGS 2k steps (identical for every instance), exact Cole–Hopf boundary data at $x = \pm 1$ —and both models are evaluated with identical metrics on the model grid. All random seeds, exact hyperparameters, generated datasets, trained weights, and metrics are committed with the code.

Reproducibility

The entire system is two packages: a zero-dependency Node.js agent ($\sim 1,600$ lines) and a PyTorch physics core ($\sim 3,300$ lines), runnable on a CPU-only laptop with Python ≥ 3.10 and Node ≥ 18 . `npm test` runs 16 agent tests; `python3 -m unittest physx.test_physx` runs 31 physics tests (47 total); both are green in the released commit. The demo (`node age.js --demo`) reproduces the end-to-end missions described in the Results section, and `npm run bench:gate` reproduces Table 5. `python physx/significance.py` reproduces the statistical tests and `python physx/deepxde_baseline.py` (with the vendored DEEPXDE wheel in `vendor/`) reproduces the external PINN comparison. `python physx/run_matrix.py` reproduces the full baseline matrix (75 runs).

Data availability

All datasets are generated deterministically from the published parameter ranges and seeds; the generator is included. Trained model weights, normalization statistics, training logs, journal files, the verification- gate benchmark results, the significance-test outputs (`paper/fig/significance.json`), the DEEPXDE comparison results (`paper/fig/deepxde_comparison.json`), and the figure data

(JSON) used to produce Figures 3–11 are included in the repository (DOI provided on acceptance).

Code availability

The complete implementation, tests, and figure-generation scripts are available at <https://github.com/placeholder/age> (DOI 10.xxxx/xxxxx provided on acceptance). The code is MIT-licensed.

Author contributions

S.R.S. conceived the system, designed and implemented the agent and the physics core, designed and ran all experiments and benchmarks, and wrote the manuscript.

Competing interests

The authors declare no competing interests. The authors have no affiliation with Prometheus or any other commercial entity named in this work.

Acknowledgements

Placeholder. We thank the open-source maintainers of PyTorch, Node.js, and the physics and agent communities whose tools made this work possible.

References

- [1] LeCun, Y., Bengio, Y. & Hinton, G. Deep learning. *Nature* **521**, 436–444 (2015).
- [2] Silver, D. *et al.* Mastering the game of Go without human knowledge. *Nature* **550**, 354–359 (2017).
- [3] Mnih, V. *et al.* Human-level control through deep reinforcement learning. *Nature* **518**, 529–533 (2015).
- [4] Brown, T. B. *et al.* Language models are few-shot learners. In *Advances in Neural Information Processing Systems 33*, 1877–1901 (2020).
- [5] Ouyang, L. *et al.* Training language models to follow instructions with human feedback. In *Advances in Neural Information Processing Systems 35* (2022).
- [6] Bubeck, S. *et al.* Sparks of artificial general intelligence: early experiments with GPT-4. Preprint at <https://arxiv.org/abs/2303.12712> (2023).
- [7] Wei, J. *et al.* Chain-of-thought prompting elicits reasoning in large language models. In *Advances in Neural Information Processing Systems 35* (2022).
- [8] Vaswani, A. *et al.* Attention is all you need. In *Advances in Neural Information Processing Systems 30*, 5998–6008 (2017).
- [9] Yao, S. *et al.* ReAct: synergizing reasoning and acting in language models. In *International Conference on Learning Representations* (2023).

- [10] Shinn, N., Cassano, F., Gopinath, A., Narasimhan, K. & Yao, S. Reflexion: language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems 36* (2023).
- [11] Wang, G. *et al.* Voyager: an open-ended embodied agent with large language models. Preprint at <https://arxiv.org/abs/2305.16291> (2023).
- [12] Park, J. S. *et al.* Generative agents: interactive simulacra of human behavior. In *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology* (2023).
- [13] Jimenez, C. E. *et al.* SWE-bench: can language models resolve real-world GitHub issues? In *International Conference on Learning Representations* (2024).
- [14] Li, Y. *et al.* Competition-level code generation with AlphaCode. *Science* **378**, 1092–1097 (2022).
- [15] Cobbe, K. *et al.* Training verifiers to solve math word problems. Preprint at <https://arxiv.org/abs/2110.14168> (2021).
- [16] Lightman, H. *et al.* Let’s verify step by step. In *International Conference on Learning Representations* (2024).
- [17] Raissi, M., Perdikaris, P. & Karniadakis, G. E. Physics-informed neural networks: a deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *J. Comput. Phys.* **378**, 686–707 (2019).
- [18] Karniadakis, G. E. *et al.* Physics-informed machine learning. *Nat. Rev. Phys.* **3**, 422–440 (2021).
- [19] Lu, L., Meng, X., Mao, Z. & Karniadakis, G. E. DeepXDE: a deep learning library for solving differential equations. *SIAM Rev.* **63**, 208–228 (2021).
- [20] Chen, R. T. Q., Rubanova, Y., Bettencourt, J. & Duvenaud, D. Neural ordinary differential equations. In *Advances in Neural Information Processing Systems 31* (2018).
- [21] Wilcoxon, F. Individual comparisons by ranking methods. *Biometrics Bull.* **1**, 80–83 (1945).
- [22] Wang, S., Teng, Y. & Perdikaris, P. Understanding and mitigating gradient flow pathologies in physics-informed neural networks. *SIAM J. Sci. Comput.* **43**, A3055–A3081 (2021).
- [23] Takamoto, M. *et al.* PDEBench: an extensive benchmark for scientific machine learning. In *Advances in Neural Information Processing Systems 35* (2022).
- [24] Kirkpatrick, J. *et al.* Overcoming catastrophic forgetting in neural networks. *Proc. Natl Acad. Sci. USA* **114**, 3521–3526 (2017).
- [25] Bommasani, R. *et al.* On the opportunities and risks of foundation models. Preprint at <https://arxiv.org/abs/2108.07258> (2021).
- [26] Sutton, R. The bitter lesson. <http://www.incompleteideas.net/IncIdeas/BitterLesson.html> (2019).
- [27] Temkin, M. Jeff Bezos’s Prometheus raises \$12B to build an ‘artificial general engineer’ for the physical world. *TechCrunch* (2026). <https://techcrunch.com/2026/06/11/jeff-bezoss-prometheus-raises-12b-to-build-an-artificial-general-engineer-for-the-physi>
- [28] Wikipedia contributors. Prometheus (company). *Wikipedia* (2025). [https://en.wikipedia.org/wiki/Prometheus_\(company\)](https://en.wikipedia.org/wiki/Prometheus_(company))